

Projet 1 : lissage du tracé de vitesse par moyenne glissante

Objectif : Avec la méthode de la moyenne glissante, lisser la courbe de vitesse.



Problème : Le tracé de la vitesse du véhicule est surprenant avec des variations de vitesse qui ne paraissent pas réelles mais dues aux incertitudes des mesures et aux calculs (la méthode d'Euler pour dériver est une approximation).

Solution : Il faut mettre en place une solution de filtrage simple appelée moyenne glissante.

Définition de la moyenne glissante appelée également moyenne mobile (Extrait Wikipédia) :

C'est une moyenne qui au lieu d'être calculée sur l'ensemble des n valeurs d'un [échantillonnage](#), est calculée tour à tour sur chaque sous-ensemble de N valeurs consécutives ($N \leq n$). Le sous-ensemble utilisé pour calculer chaque moyenne, parfois appelé « fenêtre », « glisse » sur l'ensemble des données.

Par exemple, le tableau suivant montre les moyennes mobiles simples sur 3 valeurs, pour une série de 9 mesures.

Mesures	2	3	5	8	8	7	8	5	2
Moyenne glissante	néant	$(2 + 3 + 5)/3$ 3,3333	$(3 + 5 + 8)/3$ 5,3333	$(5 + 8 + 8)/3$ 7	$(8 + 8 + 7)/3$ 7,6666	$(8 + 7 + 8)/3$ 7,6666	$(7 + 8 + 5)/3$ 6,6666	$(8 + 5 + 2)/3$ 5	néant

Autre exemple : dans le cas particulier du domaine de la pollution atmosphérique, est utilisée une « *moyenne glissante sur 8 heures* » de la concentration d'un polluant (c'est en l'occurrence le cas de l'[ozone](#), en objectif de qualité pour la protection de la santé humaine) ; cette moyenne pourra être calculée de 0h00 à 8h00, de 1h00 à 9h00, de 2h00 à 10h00, etc. On recherchera, sur une journée, la valeur maximale de la moyenne glissante, qui devra être inférieure à une concentration donnée. L'intérêt d'une moyenne glissante est de lisser les éventuels écarts accidentels.

Soit la fonction moyglissante() :

```
def moyglissante(l:[float]):
    # Calcule la moyenne glissante d'une liste de valeurs sur 3 éléments
    # Sortie: liste des valeurs moyennées (float)
    s=[None]
    for i in range(len(l)-2):
        s.append(sum(l[i:3+i])/3)
    return s
```

Q1. Ouvrez le fichier *MoyenneGlissanteEleve.py* et testez cette fonction avec l'exemple Wikipédia ci-dessus. Interprétez à l'oral.

Note : pour ouvrir un graphique dans une fenêtre Windows séparée, menu Outils, Préférences, Console Ipython, Graphiques, Sortie graphique Automatique.

Q2. La moyenne glissante peut être généralisée avec un nombre n de termes à moyenner. $n = 3$ semble être un minimum mais on peut envisager d'augmenter sa valeur pour voir l'effet sur le signal. **Ouvrez** le fichier *TP8MoyGlissanteELEVE.py*. **Testez** la fonction `moy_glissante(l:list,n:int)` sur l'exemple de Wikipédia. **Testez** l'efficacité du filtrage pour des valeurs de n supérieures à 3 sur la vitesse du véhicule.

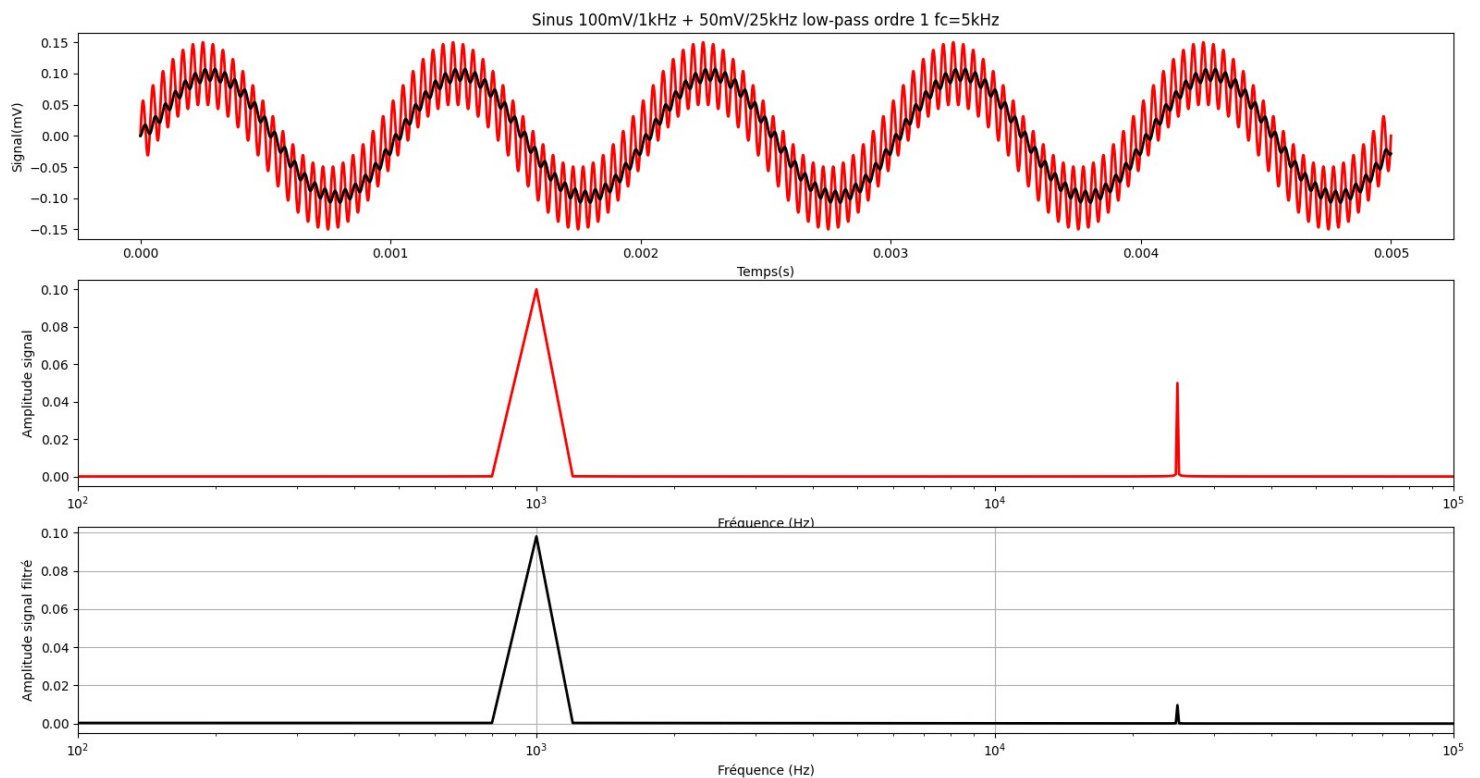
Projet 2 : Mise en œuvre d'un filtre numérique

Objectif : Implémenter un filtre numérique de type passe-bas sur un signal parasite

Données : Fichier *TP8-FiltreNumeric.py*

```
"Signal parasite en fonction du temps"
t=np.linspace(0,tmax,n) # liste de valeurs de temps en s
signal=As*np.sin(2*np.pi*Fs*t)+Ap*np.sin(2*np.pi*Fp*t)

"Calcul du signal filtré"
filtre1=np.array([0.0])
for i in range(n-1):
    filtre1=np.append(filtre1,0.030459*signal[i+1]+0.96954097*filtre1[i])
```



Q3. Testez le fichier joint. **Interprétez** à l'oral les lignes ci-dessus en précisant les types des variables utilisées.

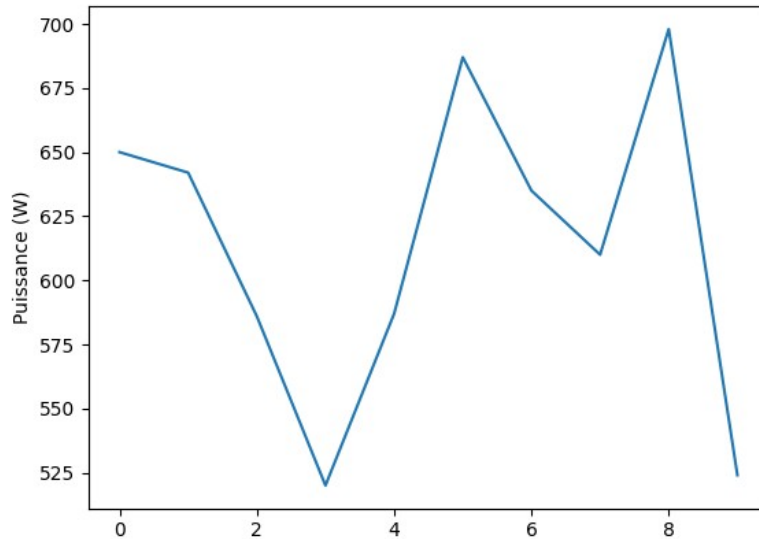
Q4. Modifiez les amplitudes et les fréquences en faisant le lien entre les chronogrammes et les spectres. **Ajoutez** un offset sur le signal et **constatez**.

Q5. En vous aidant du cours, **justifiez** l'équation de récurrence qui calcule le signal filtré à partir du signal et des caractéristiques du filtre (passe-bas, ordre1, $F_c = 5\text{kHz}$, $F_e = 1/T_e$).

Projet 3 : Détermination de l'énergie produite par un parc solaire

Objectif : Déterminer l'énergie produite à partir des valeurs de la puissance.

Données : Relevé des puissances instantanées toutes les minutes.



Solution : Pour déterminer l'énergie produite sur un intervalle de temps, il faut intégrer la puissance avec la méthode des trapèzes.

```
def trapezes(f,a:float,b:float,n:int):
    # Estimation de l'intégrale de la fonction f sur l'intervalle [a,b] sur n points
    # Entrées: f:fonction a:float,b:float,n:int
    # Sortie: s (float)
    s = 0.0
    h = (b-a)/n
    for i in range(n):
        x = a + i * h
        s = s + h*((f(x)+f(x+h))/2)
    return s
```

Q6. Afin de valider cette méthode, il est judicieux de la tester sur une solution mathématique connue. **Ouvrez** le fichier *TP8-EnergieELEVE.py* et **testez** la fonction `trapezes(f,a:float,b:float,n:int)` pour $f(x) = 3x + 2$ sur $[-1, +3]$ sur 1000 points. **Comparez** avec le résultat mathématique.

$$A = \int_{-1}^3 (3x+2) dx$$

```
def trapezesliste(p:[float],Te:float):
    # Intègre la liste p en fonction de l'intervalle de temps Te
    # Entrées: p:liste des puissances en Watt, Te: intervalle de temps en seconde
    # Sortie s:énergie en Js
    s=0.0
    for i in range(len(p)-1):
        s=s+Te*(p[i+1]+p[i])/2
    return s
```

Q7. A partir de la liste des puissances, **utilisez** la fonction `trapezesliste(p:[float],Te:float)` pour déterminer l'énergie produite en Wh. **Estimez** l'ordre de grandeur.